

Relatório Técnico de Arquitetura e Engenharia de Software

TERCEIRA ENTREGA (TP3) • CRIAÇÃO E INTEGRAÇÃO DE MICROSERVIÇO COM SPRING CLOUD

Estudante: Manoel Egídio	Instituição: Instituto Infnet
Disciplina: Engenharia de Softwares Escaláveis	Tecnologias: Spring Boot 3.4.3, Spring Cloud OpenFeign 2024, JPA, React 19, Vite
Projeto: Sistema Distribuído de Gerenciamento de Tarefas & Notificações	Data da Entrega: 26 de Setembro de 2026

1. Contexto e Objetivo da Etapa (TP3)

No escopo do curso de **Engenharia de Softwares Escaláveis**, a terceira etapa envolve a transição e expansão da aplicação monolítica anterior para uma **arquitetura distribuída baseada em microserviços**.

O objetivo central foi desacoplar as responsabilidades de comunicação, eventos e entrega de alertas, criando um **novo microserviço autônomo (notification-service)** construído com **Spring Boot** e integrado ao serviço central de tarefas (task-service) por meio de comunicação declarativa distribuída com **Spring Cloud OpenFeign**, aplicando os padrões de *Database-per-Service*, *Bounded Context* e *Graceful Degradation / Fallback*.

2. Arquitetura do Sistema Distribuído

A solução foi estruturada em dois serviços de backend independentes e uma camada de apresentação front-end moderna e reativa:

```
+-----+
+-----+ | CAMADA DE APRESENTAÇÃO | | React 19 + Vite (Porta 5173) | | - Kanban Board & Tarefas -
Central de Notificações Drawer - Monitor de Microserviços | +-----+
+-----+ | | HTTP REST /api/tasks |
HTTP REST /api/notifications v v +-----+ +-----+
+-----+ | TASK-SERVICE | | NOTIFICATION-SERVICE | | (Spring Boot
:8080) | | (Spring Boot :8082) | | | | * Bounded Context: Gestão de Tarefas | | * Bounded
Context: Notificações & Alertas | | * Spring Cloud OpenFeign: NotificationClient | | * REST
Controller: /api/notifications | | * Fallback de Resiliência: Graceful Degradation | | *
Service: NotificationServiceImpl | | * Repositórios: TaskRepository, CategoryRepo | | *
Repositório Dedicado: NotificationRepo | | * Banco Dedicado: H2 (taskdb) | | * Banco Dedicado:
H2 (notificationdb) | +-----+ +-----+
+-----+ | ^ |===== Chamada Distribuída Spring Cloud Feign =====| | POST
/api/notifications (Criação/Status) | | GET /api/notifications/task/{id} | +-----+
+-----+
```

2.1. Princípios Arquiteturais Aplicados

- **Separação de Responsabilidades (SRP / Bounded Contexts):** O ciclo de vida da tarefa reside exclusivamente no `task-service`, enquanto as regras de alerta, canais (IN_APP, EMAIL, WEBHOOK), status de leitura e histórico de destinatários residem no `notification-service`.
- **Database-per-Service:** Cada microserviço possui seu próprio banco de dados isolado (`taskdb` e `notificationdb`), impedindo acoplamento de tabelas ou joins entre serviços distintos.
- **Comunicação Declarativa com Spring Cloud OpenFeign:** A comunicação síncrona HTTP entre os serviços é realizada através de interfaces tipadas com anotações Spring MVC, eliminando boilerplate de HTTP clients.
- **Resiliência e Tolerância a Falhas (Fallback Pattern):** Caso o `notification-service` esteja temporariamente indisponível ou lento, o `NotificationClientFallback` no `task-service` absorve a falha, registrando log de contingência sem interromper o fluxo principal de tarefas do usuário.

3. Atualização do Modelo de Domínio

3.1. Modelo do Microserviço de Notificações (notification-service)

Foi modelada a entidade de domínio `Notification` contendo os seguintes atributos, índices e Value Objects:

```
@Entity @Table(name = "notifications", indexes = { @Index(name = "idx_notifications_recipient",
columnList = "recipient"), @Index(name = "idx_notifications_is_read", columnList = "is_read"),
@Index(name = "idx_notifications_task_id", columnList = "task_id"), @Index(name =
"idx_notifications_created_at", columnList = "created_at") }) public class Notification { @Id
@GeneratedValue(strategy = GenerationType.IDENTITY) private Long id; private Long taskId;
private String taskTitle; private String recipient; private String message;
@Enumerated(EnumType.STRING) private NotificationType type; // TASK_CREATED, STATUS_CHANGED,
HIGH_PRIORITY_ALERT, etc. @Enumerated(EnumType.STRING) private NotificationChannel channel; //
IN_APP, EMAIL, WEBHOOK private boolean isRead; private LocalDateTime createdAt; private
LocalDateTime readAt; // Métodos de negócio: markAsRead(), etc. }
```

3.2. Atualização do Modelo no Serviço Principal (task-service)

O serviço principal incorporou o cliente distribuído `NotificationClient` diretamente na execução de suas operações de negócio (`createTask`, `updateTask`, `updateTaskStatus` e `deleteTask`).

```
@FeignClient( name = "notification-service", url =
"${notification.service.url:http://localhost:8082}", fallback = NotificationClientFallback.class
) public interface NotificationClient { @PostMapping("/api/notifications")
NotificationResponseClientDTO sendNotification(@RequestBody NotificationPayloadDTO request);
```

```
@GetMapping("/api/notifications/task/{taskId}") List<NotificationResponseClientDTO>
getNotificationsByTaskId(@PathVariable("taskId") Long taskId); }
```

4. Endpoints da API REST

4.1. Endpoints do Microserviço de Notificações (notification-service :8082)

Método	Endpoint	Descrição	Resposta HTTP
POST	/api/notifications	Registra nova notificação a partir de eventos do sistema ou de tarefas	201 Created
GET	/api/notifications	Lista notificações com filtros opcionais por destinatário e não lidas	200 OK
GET	/api/notifications/{id}	Consulta detalhes de uma notificação por ID	200 OK
GET	/api/notifications/task/{taskId}	Retorna todos os alertas vinculados a uma tarefa específica	200 OK
GET	/api/notifications/summary	Retorna contadores consolidados (total, não lidas, lidas)	200 OK
PATCH	/api/notifications/{id}/read	Marca uma notificação específica como lida	200 OK
PATCH	/api/notifications/read-all	Marca todas as notificações pendentes como lidas em lote	200 OK
DELETE	/api/notifications/{id}	Remove uma notificação do sistema	204 No Content
POST	/api/notifications/broadcast	Emite alerta broadcast global para todos os usuários	201 Created

4.2. Novo Endpoint Distribuído no Serviço Principal (task-service :8080)

Para demonstrar a comunicação transparente entre serviços, foi adicionado um endpoint que consulta o microsserviço de notificações via OpenFeign a partir da entidade de tarefa:

```
// GET http://localhost:8080/api/tasks/{id}/notifications @GetMapping("/{id}/notifications")
public ResponseEntity<List<NotificationResponseClientDTO>> getTaskNotifications(@PathVariable
Long id) { List<NotificationResponseClientDTO> notifications =
taskService.getTaskNotifications(id); return ResponseEntity.ok(notifications); }
```

5. Repositório de Dados Dedicado (Database-per-Service)

No microsserviço notification-service, foi criado o repositório Spring Data JPA NotificationRepository, que opera exclusivamente sobre o banco de dados dedicado notificationdb:

```
@Repository public interface NotificationRepository extends JpaRepository<Notification, Long> {
List<Notification> findAllOrderByCreatedAtDesc(); List<Notification>
findByRecipientOrderByCreatedAtDesc(String recipient); List<Notification>
findByIsReadFalseOrderByCreatedAtDesc(); List<Notification>
findByRecipientAndIsReadFalseOrderByCreatedAtDesc(String recipient); List<Notification>
findByTaskIdOrderByCreatedAtDesc(Long taskId); long countByIsReadFalse(); long
countByRecipientAndIsReadFalse(String recipient); @Modifying @Query("UPDATE Notification n SET
n.isRead = true, n.readAt = CURRENT_TIMESTAMP WHERE n.recipient = :recipient AND n.isRead =
false") int markAllAsReadByRecipient(@Param("recipient") String recipient); @Modifying
@Query("UPDATE Notification n SET n.isRead = true, n.readAt = CURRENT_TIMESTAMP WHERE n.isRead =
false") int markAllAsRead(); }
```

6. Componentes Front-End Desenvolvidos

A interface de usuário em React foi ampliada com componentes visuais dedicados à interação com o microsserviço:

- **Central de Notificações (NotificationCenter.jsx):** Painel deslizante com efeito glassmorphism contendo contadores, filtros por lidas/não lidas/alertas, botões de ação em lote ("Marcar todas como lidas") e emissor de mensagens broadcast em tempo real.
- **Sino de Notificações com Badge Reativo (Navbar.jsx):** Exibe o contador de alertas não lidos em tempo real com animação de pulso luminoso (ping effect) quando chegam novos eventos.
- **Monitor de Saúde dos Microsserviços (ServiceStatusIndicator.jsx):** Exibe no cabeçalho o status de conectividade em tempo real dos serviços (task-service :8080 [Online] e notification-service :8082 [Online]).
- **Alertas Toast em Tempo Real (NotificationToast.jsx):** Notificações flutuantes que surgem instantaneamente quando ações de tarefas emitem eventos ao microsserviço.

- **Modal de Notificações por Tarefa (TaskNotificationModal.jsx):** Botão de sino em cada cartão Kanban permitindo inspecionar a rota distribuída de alertas associados àquela tarefa.

7. Cobertura de Testes Automatizados

Foram desenvolvidas suítes de testes unitários e de integração cobrindo os repositórios, serviços de negócio, controladores REST e clientes distribuídos de ambos os componentes:

1. Suíte de Testes do notification-service:

BUILD SUCCESSFUL in 13s • 5 actionable tasks: 3 executed, 2 up-to-date (100% dos 7 testes passaram)

- NotificationRepositoryTest (Persistência, busca de não-lidas, contagem e atualização em lote)
- NotificationServiceTest (Criação de notificações, marcação de leitura, cálculo de resumo)
- NotificationControllerTest (Validação dos endpoints POST, GET, PATCH e DELETE via HTTP)

2. Suíte de Testes do task-service:

BUILD SUCCESSFUL in 20s • 5 actionable tasks: 3 executed, 2 up-to-date (100% dos 11 testes passaram)

- NotificationIntegrationTest (Injeção de Feign Client, execução de Fallback com resiliência, integração com ciclo de tarefas)
- TaskManagerApplicationTests (Carga de contexto, rastreabilidade e histórico)
- TaskRepositoryTest & TaskHistoryRepositoryTest (Consultas JPQL e paginação)

8. Atendimento Integral à Rubrica de Avaliação

Item da Rubrica (TP3)	Evidência Técnica Implementada	Status
O aluno incorporou um novo serviço no modelo de domínio refletindo mudanças na arquitetura?	Modelo de domínio atualizado com entidades Notification, tipos e integração distribuída no TaskServiceImpl.	Demonstrou o item de rubrica
O aluno implementou novos endpoints para acesso ao microserviço via REST API?	Implementados 9 novos endpoints REST em /api/notifications e endpoint distribuído /api/tasks/{id}/notifications.	Demonstrou o item de rubrica
O aluno usou Spring Boot para o desenvolvimento do microserviço?	Microserviço autônomo desenvolvido com Spring Boot 3.4.3 rodando na porta 8082.	Demonstrou o item de rubrica

Item da Rubrica (TP3)	Evidência Técnica Implementada	Status
O aluno aplicou Spring Cloud para facilitar a configuração e comunicação distribuídas?	Utilizado Spring Cloud OpenFeign com clientes declarativos, configuração externalizada e Fallback de resiliência.	Demonstrou o item de rubrica
O aluno criou um novo repositório de dados dedicados para gerenciar dados específicos do microserviço?	Criado NotificationRepository operando sobre base isolada (notificationdb), garantindo o padrão <i>Database-per-Service</i> .	Demonstrou o item de rubrica
O aluno adicionou componentes na interface do usuário para interação com o novo microserviço?	Adicionados NotificationCenter, sino no navbar com contador em tempo real, toasters, e modal de alertas por tarefa.	Demonstrou o item de rubrica
O aluno expandiu a cobertura de testes para incluir o novo microserviço e atualizações no sistema?	Cobertura expandida com 18 testes automatizados executados com 100% de sucesso entre os dois microserviços.	Demonstrou o item de rubrica
O aluno apresentou o código-fonte do microserviço e atualizações correspondentes no sistema e interface de usuário?	Código-fonte integralmente estruturado na pasta TP3/ (task-service, notification-service, frontend).	Demonstrou o item de rubrica
O aluno apresentou a documentação arquitetural detalhada do microserviço, integração com o sistema, e descrição dos novos endpoints da API?	Documentação completa neste relatório técnico contendo diagramas, justificativas, mapeamento de rotas e dados de teste.	Demonstrou o item de rubrica
O aluno fez uma apresentação prática mostrando a operação integrada do novo microserviço?	Fluxo integrado validado de ponta a ponta com chamadas REST entre os dois serviços e interface web conectada.	Demonstrou o item de rubrica

9. Guia de Execução Local

```
# 1. Executar o Microserviço de Notificações (Spring Boot na porta 8082) cd TP3/notification-service ./gradlew bootRun # 2. Executar o Microserviço Principal de Tarefas (Spring Boot na porta 8080) cd ../task-service ./gradlew bootRun # 3. Executar os Testes Automatizados de Ambos os Serviços cd ../notification-service && ./gradlew test cd ../task-service && ./gradlew test #
```

```
4. Executar a Interface do Usuário (React + Vite na porta 5173) cd ../frontend bun install bun
run dev # 5. Acessar a aplicação integrada Frontend: http://localhost:5173 API Task Service:
http://localhost:8080/api/tasks API Notification Service:
http://localhost:8082/api/notifications
```